

■ CNN Image Dataset

Complete Practical Loading Guide

TensorFlow / Keras — All 12 Methods with Deep Code Examples

Beginner → Intermediate → Advanced → Industry

Topics Covered

#	Method	Use-Case
1	<code>image_dataset_from_directory()</code>	Simplest approach for structured folders
2	Train/Test Folder Structure	Standard Kaggle-style split
3	CSV-Based Dataset Loading	Medical / enterprise datasets
4	OpenCV Loading	Production-grade pixel-level control
5	PIL / Pillow Loading	Lightweight preprocessing
6	NumPy Array Loading	Pre-processed arrays in memory
7	TensorFlow Datasets (TFDS)	Ready-made benchmark datasets
8	Kaggle API	Download & use Kaggle datasets
9	Google Drive	Colab + large remote datasets
10	<code>ImageDataGenerator</code>	On-the-fly augmentation pipeline
11	<code>tf.data.Dataset</code>	Scalable production pipeline
12	Comparison & Recommendations	When to use what

01.

image_dataset_from_directory()

[BEGINNER | Keras Built-in]

What it does: Keras ka built-in function jo ek structured folder se automatically image dataset load karta hai. Folder ke naam se labels automatically assign hote hain.

Required Folder Structure:

```
dataset/  
├── cats/  
│   ├── cat001.jpg  
│   ├── cat002.jpg  
│   └── ...  
└── dogs/  
    ├── dog001.jpg  
    ├── dog002.jpg  
    └── ...
```

Basic Example:

```
import tensorflow as tf  
  
# Basic loading  
train_data = tf.keras.utils.image_dataset_from_directory(  
    'dataset',  
    image_size=(224, 224), # Resize all images  
    batch_size=32, # 32 images per batch  
    seed=42 # Reproducibility ke liye  
)  
  
# Class names check karo  
print('Classes:', train_data.class_names) # ['cats', 'dogs']
```

Advanced Example — Validation Split ke Saath:

```
# Training aur Validation split automatically
train_data = tf.keras.utils.image_dataset_from_directory(
    'dataset',
    validation_split=0.2, # 20% validation
    subset='training',
    seed=42,
    image_size=(224, 224),
    batch_size=32,
    label_mode='int', # 'int', 'categorical', 'binary'
    color_mode='rgb', # 'rgb' ya 'grayscale'
    shuffle=True
)

val_data = tf.keras.utils.image_dataset_from_directory(
    'dataset',
    validation_split=0.2,
    subset='validation',
    seed=42,
    image_size=(224, 224),
    batch_size=32
)
```

Performance Optimization (Prefetch + Cache):

```
AUTOTUNE = tf.data.AUTOTUNE

train_data = train_data
    .cache() # Data RAM me store karo
    .shuffle(1000) # Buffer shuffle
    .prefetch(buffer_size=AUTOTUNE) # GPU wait na kare

val_data = val_data.prefetch(buffer_size=AUTOTUNE)

# Model training
model.fit(train_data, validation_data=val_data, epochs=10)
```

■ Advantages:

- Beginner-friendly, minimum code
- Automatic label creation from folder names
- Built-in validation split support
- Returns `tf.data.Dataset` (scalable)

■ Limitations:

- Only folder-based structure support
- Custom preprocessing limited

02.

Train / Test Folder Structure

[BEGINNER-INTERMEDIATE | Kaggle Standard]

Kaggle competitions mein yeh structure sabse common hai. Train aur Test alag folders mein hote hain.

Folder Structure:

```
dataset/  
  train/  
    cats/  
      cat001.jpg ...  
    dogs/  
      dog001.jpg ...  
  test/  
    cats/  
    dogs/
```

Complete Loading Code:

```

import tensorflow as tf

IMG_SIZE = (224, 224)
BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE

# Train data load karo
train_data = tf.keras.utils.image_dataset_from_directory(
    'dataset/train',
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=True,
    seed=42
)

# Test data load karo (no shuffle for testing)
test_data = tf.keras.utils.image_dataset_from_directory(
    'dataset/test',
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False
)

# Normalization layer
normalization_layer = tf.keras.layers.Rescaling(1./255)

train_data = train_data.map(lambda x, y: (normalization_layer(x), y))
.cache().shuffle(500).prefetch(AUTOTUNE)

test_data = test_data.map(lambda x, y: (normalization_layer(x), y))
.prefetch(AUTOTUNE)

# Dataset info
for images, labels in train_data.take(1):
    print('Image batch shape:', images.shape) # (32, 224, 224, 3)
    print('Label batch shape:', labels.shape) # (32,)

```

Manual Train/Val/Test Split (3-way):

```

# Agar sirf train folder hai to manually split karo
full_train = tf.keras.utils.image_dataset_from_directory(
    'dataset/train',
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    seed=42
)

total_batches = tf.data.experimental.cardinality(full_train).numpy()
train_size = int(0.8 * total_batches) # 80% train
val_size = int(0.2 * total_batches) # 20% val

train_ds = full_train.take(train_size)
val_ds = full_train.skip(train_size).take(val_size)

```

03.

CSV-Based Dataset Loading

[INTERMEDIATE | Medical / Custom Datasets]

Jab image paths aur labels ek CSV file mein hoon, tab yeh approach use karte hain. Medical imaging, enterprise datasets mein bahut common hai.

CSV Format (labels.csv):

```
image_path,label,class_name
images/cat001.jpg,0,cat
images/dog001.jpg,1,dog
images/cat002.jpg,0,cat
images/dog002.jpg,1,dog
```

Method A — pandas + PIL (Simple):

```
import pandas as pd
import numpy as np
from PIL import Image
import tensorflow as tf

# CSV padhlo
df = pd.read_csv('labels.csv')
print(df.head())
print('Total images:', len(df))

IMG_SIZE = (224, 224)

def load_image_pil(path, label):
    img = Image.open(path).convert('RGB')
    img = img.resize(IMG_SIZE)
    img = np.array(img, dtype=np.float32) / 255.0
    return img, label

# Sab images load karo
X, y = [], []
for _, row in df.iterrows():
    img, label = load_image_pil(row['image_path'], row['label'])
    X.append(img)
    y.append(label)

X = np.array(X) # (N, 224, 224, 3)
y = np.array(y) # (N,)
print('Dataset shape:', X.shape)
```

Method B — tf.data Pipeline (Scalable):

```

import pandas as pd
import tensorflow as tf

df = pd.read_csv('labels.csv')

# TF dataset banaao from CSV
file_paths = df['image_path'].values
labels = df['label'].values

def parse_image(file_path, label):
    img = tf.io.read_file(file_path)
    img = tf.image.decode_jpeg(img, channels=3) # ya decode_png
    img = tf.image.resize(img, [224, 224])
    img = tf.cast(img, tf.float32) / 255.0
    return img, label

dataset = tf.data.Dataset.from_tensor_slices((file_paths, labels))
dataset = dataset.map(parse_image, num_parallel_calls=tf.data.AUTOTUNE)
dataset = dataset.batch(32).prefetch(tf.data.AUTOTUNE)

```

Multi-class CSV with One-Hot Encoding:

```

# Multi-class ke liye
num_classes = df['label'].nunique()

def parse_multiclass(file_path, label):
    img = tf.io.read_file(file_path)
    img = tf.image.decode_image(img, channels=3, expand_animations=False)
    img = tf.image.resize(img, [224, 224])
    img = tf.cast(img, tf.float32) / 255.0
    label_onehot = tf.one_hot(label, depth=num_classes)
    return img, label_onehot

dataset = tf.data.Dataset.from_tensor_slices((file_paths, labels))
dataset = dataset.map(parse_multiclass, num_parallel_calls=tf.data.AUTOTUNE)
dataset = dataset.shuffle(1000).batch(32).prefetch(tf.data.AUTOTUNE)

```

04.

OpenCV Loading (cv2)

[INTERMEDIATE | Production Computer Vision]

OpenCV sabse powerful image processing library hai. BGR format use karta hai, isliye RGB convert karna yaad rakho. Production systems mein widely used.

Installation:

```
pip install opencv-python
```

Basic OpenCV Loading:

```
import cv2
import numpy as np

# Image load karo (BGR format mein aata hai!)
img_bgr = cv2.imread('cat.jpg')

# BGR → RGB convert karo (important!)
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Resize
img_resized = cv2.resize(img_rgb, (224, 224))

# Normalize [0,1]
img_norm = img_resized.astype(np.float32) / 255.0

print('Shape:', img_norm.shape) # (224, 224, 3)
print('Dtype:', img_norm.dtype) # float32
print('Min/Max:', img_norm.min(), img_norm.max())
```

Folder se Batch Loading:


```

import cv2, os, numpy as np

def load_dataset_opencv(folder_path, img_size=(224,224)):
    X, y = [], []
    class_names = sorted(os.listdir(folder_path))
    class_to_idx = {cls: i for i, cls in enumerate(class_names)}

    for class_name in class_names:
        class_path = os.path.join(folder_path, class_name)
        if not os.path.isdir(class_path): continue

        for img_file in os.listdir(class_path):
            if not img_file.lower().endswith(('.jpg', '.jpeg', '.png')):
                continue
            img_path = os.path.join(class_path, img_file)
            img = cv2.imread(img_path)
            if img is None: continue # corrupted skip
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            img = cv2.resize(img, img_size)
            img = img.astype(np.float32) / 255.0
            X.append(img)
            y.append(class_to_idx[class_name])

    return np.array(X), np.array(y), class_names

X_train, y_train, classes = load_dataset_opencv('dataset/train')
print(f'Loaded: {X_train.shape}, Classes: {classes}')

```

Advanced — CLAHE + Preprocessing Pipeline:

```

def preprocess_medical_image(img_path, img_size=(224,224)):
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

    # CLAHE – contrast enhancement
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    img = clahe.apply(img)

    # Denoise
    img = cv2.fastNlMeansDenoising(img, h=10)

    # Resize
    img = cv2.resize(img, img_size)

    # 3-channel banana (CNNs ke liye)
    img = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)
    img = img.astype(np.float32) / 255.0
    return img

```

■■ Important: OpenCV BGR vs RGB

- cv2.imread() → BGR format (Blue, Green, Red)
- Matplotlib/TensorFlow → RGB format expect karte hain
- Hamesha cvtColor(img, cv2.COLOR_BGR2RGB) karo visualization se pehle

05.

PIL / Pillow Loading

[BEGINNER | Simple Preprocessing]

PIL (Python Imaging Library) ya Pillow ek lightweight aur simple image loading library hai. OpenCV se simpler hai aur directly RGB format deta hai.

Installation:

```
pip install Pillow
```

Basic PIL Loading:

```
from PIL import Image
import numpy as np

# Image open karo
img = Image.open('cat.jpg')
print('Mode:', img.mode) # RGB, RGBA, L (grayscale)
print('Size:', img.size) # (width, height) – Note: width first!

# RGB ensure karo
img = img.convert('RGB')

# Resize
img = img.resize((224, 224), Image.LANCZOS) # Best quality

# NumPy array
img_array = np.array(img, dtype=np.float32) / 255.0
print('Array shape:', img_array.shape) # (224, 224, 3)
```

Augmentation with PIL:

```

from PIL import Image, ImageEnhance, ImageFilter
import random, numpy as np

def augment_pil(img_path):
    img = Image.open(img_path).convert('RGB')

    # Random horizontal flip
    if random.random() > 0.5:
        img = img.transpose(Image.FLIP_LEFT_RIGHT)

    # Random rotation ±15 degrees
    angle = random.uniform(-15, 15)
    img = img.rotate(angle, fillcolor=(0,0,0))

    # Brightness adjust
    factor = random.uniform(0.8, 1.2)
    img = ImageEnhance.Brightness(img).enhance(factor)

    # Contrast adjust
    factor = random.uniform(0.8, 1.2)
    img = ImageEnhance.Contrast(img).enhance(factor)

    img = img.resize((224, 224))
    return np.array(img, dtype=np.float32) / 255.0

# Use karo
augmented = augment_pil('cat.jpg')
print('Augmented shape:', augmented.shape)

```

PIL se tf.data Pipeline:

```

import tensorflow as tf
from PIL import Image
import numpy as np

def load_with_pil(path, label):
    # tf.py_function use karo PIL ke saath
    def _load(p):
        img = Image.open(p.numpy().decode()).convert('RGB')
        img = img.resize((224, 224))
        return np.array(img, dtype=np.float32) / 255.0

    img = tf.py_function(_load, [path], tf.float32)
    img.set_shape([224, 224, 3])
    return img, label

paths = ['cat.jpg', 'dog.jpg']
labels = [0, 1]
ds = tf.data.Dataset.from_tensor_slices((paths, labels))
ds = ds.map(load_with_pil).batch(32).prefetch(tf.data.AUTOTUNE)

```

06.

NumPy Array Loading

[INTERMEDIATE | Pre-processed Data]

Jab images already numpy arrays mein stored hain (jaise .npy, .npz files, ya MNIST jaise datasets), tab directly model ko feed kar sakte hain.

Save aur Load NumPy Arrays:

```
import numpy as np

# Pehle se processed data save karna
X_train = np.random.rand(1000, 224, 224, 3).astype(np.float32)
y_train = np.random.randint(0, 2, 1000)

# Save karo
np.save('X_train.npy', X_train)
np.save('y_train.npy', y_train)

# Compressed save (NPZ)
np.savez_compressed('dataset.npz', X=X_train, y=y_train)

# Load karo
X_train = np.load('X_train.npy')
y_train = np.load('y_train.npy')

# NPZ load
data = np.load('dataset.npz')
X_train = data['X']
y_train = data['y']
print('Loaded X shape:', X_train.shape) # (1000, 224, 224, 3)
```

Directly Model ko Feed karo:

```

import tensorflow as tf
import numpy as np

# Data load karo
X = np.load('X_train.npy') # (N, H, W, C) – float32
y = np.load('y_train.npy') # (N,) – int

# Verify shapes
assert X.dtype == np.float32, 'Float32 hona chahiye!'
assert X.min() >= 0.0 and X.max() <= 1.0, 'Normalize karo!'

# Method 1: Direct fit
model.fit(X, y, batch_size=32, epochs=10, validation_split=0.2)

# Method 2: tf.data se (better for large data)
dataset = tf.data.Dataset.from_tensor_slices((X, y))
dataset = dataset.shuffle(1000).batch(32).prefetch(tf.data.AUTOTUNE)
model.fit(dataset, epochs=10)

```

MNIST Example (Classic NumPy Dataset):

```

from tensorflow.keras.datasets import mnist, cifar10
import numpy as np

# MNIST load karo
(X_train, y_train), (X_test, y_test) = mnist.load_data()
print('MNIST Train:', X_train.shape) # (60000, 28, 28)

# CNN ke liye reshape (channels add karo)
X_train = X_train.reshape(-1, 28, 28, 1).astype(np.float32) / 255.0
X_test = X_test.reshape(-1, 28, 28, 1).astype(np.float32) / 255.0

# CIFAR-10
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
print('CIFAR-10 Train:', X_train.shape) # (50000, 32, 32, 3)
X_train = X_train.astype(np.float32) / 255.0

```

07.

TensorFlow Datasets (TFDS)

[BEGINNER | Benchmark Datasets]

TFDS pre-built, ready-to-use datasets provide karta hai. Research aur learning ke liye perfect.

Installation:

```
pip install tensorflow-datasets
```

Basic TFDS Loading:

```
import tensorflow_datasets as tfds
import tensorflow as tf

# Available datasets dekhna
print(tfds.list_builders()[10])

# Cats vs Dogs dataset
dataset, info = tfds.load(
    'cats_vs_dogs',
    split=['train[:80%]', 'train[80%:]'], # 80/20 split
    with_info=True,
    as_supervised=True # (image, label) format
)

train_ds, val_ds = dataset
print(info.features) # Dataset info
print(info.splits['train'].num_examples)
```

Preprocessing + Pipeline:

```

IMG_SIZE = 224
BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE

def preprocess(image, label):
    image = tf.image.resize(image, [IMG_SIZE, IMG_SIZE])
    image = tf.cast(image, tf.float32) / 255.0
    return image, label

def augment(image, label):
    image = tf.image.random_flip_left_right(image)
    image = tf.image.random_brightness(image, 0.2)
    image = tf.image.random_contrast(image, 0.8, 1.2)
    return image, label

train_ds = (train_ds
    .map(preprocess, num_parallel_calls=AUTOTUNE)
    .map(augment, num_parallel_calls=AUTOTUNE)
    .cache()
    .shuffle(1000)
    .batch(BATCH_SIZE)
    .prefetch(AUTOTUNE)
)

val_ds = (val_ds
    .map(preprocess, num_parallel_calls=AUTOTUNE)
    .batch(BATCH_SIZE)
    .prefetch(AUTOTUNE)
)

```

Popular TFDS Datasets:

```

# ImageNet (large, requires approval)
ds = tfds.load('imagenet2012', split='train')

# Oxford Flowers
ds = tfds.load('oxford_flowers102', as_supervised=True)

# Stanford Dogs
ds = tfds.load('stanford_dogs', as_supervised=True)

# Rock Paper Scissors
ds = tfds.load('rock_paper_scissors', as_supervised=True)

# CIFAR-100
ds = tfds.load('cifar100', as_supervised=True)

```

08.

Kaggle API

[INTERMEDIATE | Competition Datasets]

Kaggle API se directly datasets download kar sakte hain — Google Colab mein sabse common workflow.

Setup (Google Colab):

```
# Step 1: kaggle.json upload karo (Kaggle Account > API > Create Token)
from google.colab import files
files.upload() # kaggle.json select karo

# Step 2: File move karo
import os, shutil
os.makedirs('/root/.kaggle', exist_ok=True)
shutil.move('kaggle.json', '/root/.kaggle/')
os.chmod('/root/.kaggle/kaggle.json', 600) # Permissions

# Step 3: Kaggle install karo
# pip install kaggle (already in Colab)
```

Dataset Download:

```
# Kaggle competition dataset
!kaggle competitions download -c dogs-vs-cats

# Kaggle dataset (not competition)
!kaggle datasets download -d salader/dogs-vs-cats

# Unzip karo
!unzip -q dogs-vs-cats.zip -d /content/dataset/

# Folder structure check karo
import os
for root, dirs, files in os.walk('/content/dataset'):
    level = root.replace('/content/dataset', '').count(os.sep)
    indent = ' ' * 2 * level
    print(f'{indent}{os.path.basename(root)}/')
    if level < 2:
        subindent = ' ' * 2 * (level + 1)
        print(f'{subindent}Files: {len(files)}')
```

Full Kaggle → Train Pipeline:


```
import tensorflow as tf

# Download → Unzip → Load
!kaggle datasets download -d salader/dogs-vs-cats
!unzip -q dogs-vs-cats.zip -d /content/data/

train_ds = tf.keras.utils.image_dataset_from_directory(
    '/content/data/train',
    image_size=(224, 224),
    batch_size=32,
    validation_split=0.2,
    subset='training',
    seed=42
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    '/content/data/train',
    image_size=(224, 224),
    batch_size=32,
    validation_split=0.2,
    subset='validation',
    seed=42
)

AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
```

09.

Google Drive Loading

[BEGINNER | Colab + Large Datasets]

Bade datasets Google Drive mein store karke Colab se access kar sakte hain. Internet bandwidth save hoti hai bar-bar download na karna pade.

Drive Mount karo:

```
from google.colab import drive
drive.mount('/content/drive')
# Browser mein Google account authorize karo

# Drive path
DRIVE_PATH = '/content/drive/MyDrive/datasets/cats_dogs'

import os
print('Files:', os.listdir(DRIVE_PATH))
```

Direct Loading from Drive:

```
import tensorflow as tf

# Seedha Drive se load karo
train_ds = tf.keras.utils.image_dataset_from_directory(
    '/content/drive/MyDrive/datasets/cats_dogs/train',
    image_size=(224, 224),
    batch_size=32
)

# Drive slow hota hai – pehle copy karo local
import shutil
shutil.copytree(
    '/content/drive/MyDrive/datasets/cats_dogs',
    '/content/cats_dogs'
)

# Ab local se load karo (fast!)
train_ds = tf.keras.utils.image_dataset_from_directory(
    '/content/cats_dogs/train',
    image_size=(224, 224),
    batch_size=32
)
```

ZIP file from Drive:

```
import zipfile

# Drive mein ZIP hai
zip_path = '/content/drive/MyDrive/datasets/cats_dogs.zip'

# Extract karo
with zipfile.ZipFile(zip_path, 'r') as zf:
    zf.extractall('/content/dataset')

print('Extracted!')
```

■ Tip: Drive I/O slow hai. Training ke liye hamesha local /content/ mein copy karo!

10.

ImageDataGenerator

[INTERMEDIATE | Augmentation + Loading]

ImageDataGenerator purani TF/Keras approach hai — load aur augment simultaneously karta hai. Older code mein common hai, lekin tf.data se slower hai.

Basic ImageDataGenerator:

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Training ke liye augmentation define karo
train_datagen = ImageDataGenerator(
    rescale=1./255, # Normalize [0,1]
    rotation_range=20, # ±20 degree rotation
    width_shift_range=0.2, # Horizontal shift
    height_shift_range=0.2, # Vertical shift
    shear_range=0.2, # Shear transform
    zoom_range=0.2, # Zoom in/out
    horizontal_flip=True, # Mirror flip
    vertical_flip=False, # Vertical flip (usually False)
    fill_mode='nearest' # Border fill method
)

# Validation ke liye sirf normalize (no augment!)
val_datagen = ImageDataGenerator(rescale=1./255)
```

flow_from_directory:

```

train_gen = train_datagen.flow_from_directory(
    'dataset/train',
    target_size=(224, 224), # Resize
    batch_size=32,
    class_mode='binary', # 'binary', 'categorical', 'sparse'
    shuffle=True,
    seed=42
)

val_gen = val_datagen.flow_from_directory(
    'dataset/val',
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary',
    shuffle=False
)

print('Classes:', train_gen.class_indices) # {'cats':0, 'dogs':1}

# Model fit
model.fit(
    train_gen,
    steps_per_epoch=train_gen.samples // train_gen.batch_size,
    validation_data=val_gen,
    validation_steps=val_gen.samples // val_gen.batch_size,
    epochs=10
)

```

flow_from_dataframe (CSV ke saath):

```
import pandas as pd

df = pd.read_csv('labels.csv')
df['label'] = df['label'].astype(str) # String banana zaroori!

gen = ImageDataGenerator(rescale=1./255, horizontal_flip=True)

train_gen = gen.flow_from_dataframe(
    df,
    x_col='image_path', # Image path column
    y_col='label', # Label column
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical'
)

# Visualize augmented images
import matplotlib.pyplot as plt
batch = next(train_gen)
fig, axes = plt.subplots(2, 4, figsize=(12, 6))
for i, ax in enumerate(axes.flat):
    ax.imshow(batch[0][i])
    ax.set_title(f'Label: {batch[1][i].argmax()}')
    ax.axis('off')
plt.show()
```

11.

tf.data.Dataset Pipeline

[ADVANCED | Production-Grade]

tf.data sabse scalable aur production-ready approach hai. GPU/TPU training ke liye optimal. Large datasets ke liye yahi best choice hai.

Basic tf.data Pipeline:

```
import tensorflow as tf
import pathlib

# Image paths aur labels collect karo
data_dir = pathlib.Path('dataset')
class_names = sorted([d.name for d in data_dir.iterdir() if d.is_dir()])

all_paths, all_labels = [], []
for label, cls in enumerate(class_names):
    for path in (data_dir / cls).glob('*.jpg'):
        all_paths.append(str(path))
        all_labels.append(label)

print(f'Total images: {len(all_paths)}')
print(f'Classes: {class_names}')

# tf.data dataset banao
path_ds = tf.data.Dataset.from_tensor_slices(all_paths)
label_ds = tf.data.Dataset.from_tensor_slices(all_labels)
dataset = tf.data.Dataset.zip((path_ds, label_ds))
```

Complete Parse + Augment Pipeline:

```

IMG_SIZE = 224
BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE

def parse_image(path, label):
    img = tf.io.read_file(path)
    img = tf.image.decode_jpeg(img, channels=3)
    img = tf.image.resize(img, [IMG_SIZE, IMG_SIZE])
    img = tf.cast(img, tf.float32) / 255.0
    return img, label

def augment_train(img, label):
    img = tf.image.random_flip_left_right(img)
    img = tf.image.random_brightness(img, max_delta=0.2)
    img = tf.image.random_contrast(img, 0.8, 1.2)
    img = tf.image.random_saturation(img, 0.8, 1.2)
    img = tf.image.random_hue(img, 0.1)
    img = tf.clip_by_value(img, 0.0, 1.0) # Clamp values
    return img, label

# Train pipeline
train_ds = (dataset
    .shuffle(len(all_paths), seed=42) # Full shuffle
    .map(parse_image, num_parallel_calls=AUTOTUNE)
    .map(augment_train, num_parallel_calls=AUTOTUNE)
    .batch(BATCH_SIZE, drop_remainder=True)
    .cache() # RAM me cache (faster epochs)
    .prefetch(AUTOTUNE)# GPU pehle se data ready rakhe
)

```

Interleave — Parallel File Reading:


```
# Multiple shards/folders se parallel read karo
file_pattern = 'dataset/**/*.jpg'
files = tf.data.Dataset.list_files(file_pattern, shuffle=True)

def process_path(file_path):
    parts = tf.strings.split(file_path, os.sep)
    class_name = parts[-2]
    label = tf.cast(
        tf.where(class_name == 'cats', 0, 1),
        tf.int32
    )
    img = tf.io.read_file(file_path)
    img = tf.image.decode_jpeg(img, channels=3)
    img = tf.image.resize(img, [224, 224])
    img = tf.cast(img, tf.float32) / 255.0
    return img, label

dataset = files.interleave(
    lambda x: tf.data.Dataset.from_tensors(x).map(process_path),
    cycle_length=4,
    num_parallel_calls=AUTOTUNE
)
```

TFRecord Format (Ultimate Performance):

```

# TFRecord banao – serialized binary format (fastest I/O)
def serialize_example(image_raw, label):
    feature = {
        'image': tf.train.Feature(
            bytes_list=tf.train.BytesList(value=[image_raw])),
        'label': tf.train.Feature(
            int64_list=tf.train.Int64List(value=[label])),
    }
    return tf.train.Example(
        features=tf.train.Features(feature=feature)).SerializeToString()

# Write TFRecord
with tf.io.TFRecordWriter('train.tfrecord') as writer:
    for path, label in zip(all_paths, all_labels):
        image_raw = open(path, 'rb').read()
        writer.write(serialize_example(image_raw, label))

# Read TFRecord
def parse_tfrecord(serialized):
    feature_desc = {
        'image': tf.io.FixedLenFeature([], tf.string),
        'label': tf.io.FixedLenFeature([], tf.int64),
    }
    example = tf.io.parse_single_example(serialized, feature_desc)
    image = tf.image.decode_jpeg(example['image'], channels=3)
    image = tf.image.resize(image, [224, 224])
    image = tf.cast(image, tf.float32) / 255.0
    return image, example['label']

tfrecord_ds = tf.data.TFRecordDataset('train.tfrecord')
tfrecord_ds = tfrecord_ds.map(parse_tfrecord, num_parallel_calls=AUTOTUNE)
tfrecord_ds = tfrecord_ds.batch(32).prefetch(AUTOTUNE)

```

12.

Comparison & Recommendations

[GUIDE | When to Use What]

Sabhi methods ka detailed comparison — kab kya use karna chahiye:

Method	Level	Speed	Augment	Best For
image_dataset_from_directory	Beginner	Fast	No	Personal projects, learning
Train/Test Folder	Beginner	Fast	No	Kaggle competitions
CSV-Based	Intermediate	Medium	Custom	Medical, enterprise data
OpenCV	Intermediate	Fast	Custom	Production CV systems
PIL/Pillow	Beginner	Medium	Yes	Simple preprocessing
NumPy Arrays	Intermediate	Very Fast	No	Pre-processed data
TFDS	Beginner	Fast	Custom	Research benchmarks
Kaggle API	Intermediate	Medium	No	Competition datasets
Google Drive	Beginner	Slow	No	Colab large datasets
ImageDataGenerator	Intermediate	Medium	Built-in	Older TF projects
tf.data.Dataset	Advanced	Fastest	Custom	Production, large scale

Recommended Paths:

■ Beginner	image_dataset_from_directory + prefetch + cache
■ Intermediate	OpenCV loading + NumPy arrays + tf.data
■ Advanced	tf.data.Dataset with TFRecords + augmentation pipeline
■ Industry	OpenCV + tf.data + TFRecord + TPU-compatible pipeline

Golden Rules:

1. Hamesha data normalize karo — images [0, 1] ya [-1, 1] range mein hone chahiye.
2. Training pe augmentation karo, validation/test pe NEVER.
3. prefetch(AUTOTUNE) hamesha lagao — GPU ko wait nahi karna chahiye.
4. Bade datasets ke liye TFRecord format use karo — I/O fastest hota hai.
5. cache() tab use karo jab dataset RAM mein fit ho.
6. OpenCV BGR → RGB convert karna mat bhoolo!
7. Dataset ki shape (N, H, W, C) verify karo training se pehle.